

HW 3

Sam Ly

February 14, 2026

Chapter 6

The code for the challenge questions can be found here:

<https://github.com/samlyme/Assignments/tree/cs4080/ch6-challenge>

1. In C, a block is a statement form that allows you to pack a series of statements where a single one is expected. The comma operator is an analogous syntax for expressions. A comma-separated series of expressions can be given where a single expression is expected (except inside a function call's argument list). At runtime, the comma operator evaluates the left operand and discards the result. Then it evaluates and returns the right operand.

Add support for comma expressions. Give them the same precedence and associativity as in C. Write the grammar, and then implement the necessary parsing code.

First, to create the grammar, we define a new lowest precedence below `equality`. So, the new grammar becomes:

```
expression    → comma;      // CHALLENGE!!
comma         →> equality ("," equality)*;
equality      → comparison ( ( "!=" | "==" ) comparison )* ;
comparison    → term ( ( ">" | ">=" | "<" | "<=" ) term )* ;
term          → factor ( ( "-" | "+" ) factor )* ;
factor        → unary ( ( "/" | "*" ) unary )* ;
unary         → ( "!" | "-" ) unary
              | primary ;
primary       → NUMBER | STRING | "true" | "false" | "nil"
              | "(" expression ")" ;
```

To implement this, we override the current `Expr` method to match for the `Comma` token first.

```
private Expr expression() {
    return comma();
}

private Expr comma() {
    Expr expr = equality();

    while (match(COMMA)) {
        Token op = previous();
        Expr right = equality();
        expr = new Expr.Binary(expr, op, right);
    }
    return expr;
}
```

- Likewise, add support for the C-style conditional or “ternary” operator `?:`. What precedence level is allowed between the `?` and `:`? Is the whole operator left-associative or right-associative?

The ternary operator has slightly higher precedence than the comma. This is because `(a ? b : c, d, e)` should evaluate to `(b, d, e)` or `(c, d, e)`.

Although the C-style ternary can be nested and is right-associative, in my implementation I chose to explicitly enforce the use of parens when nesting ternaries. The implementation also required creating a new Expression type since the ternary expression is actually a trinary expression.

```
private Expr ternary() {
    Expr expr = equality();

    if (match(QUESTION)) {
        Token leftOp = previous();
        Expr center = equality();

        consume(COLON, "Expect ':' in ternary.");
        Token rightOp = previous();
        Expr right = equality();

        expr = new Expr.Trinary(expr, leftOp, center, rightOp, right);
    }

    return expr;
}
```

- Add error productions to handle each binary operator appearing without a left-hand operand. In other words, detect a binary operator appearing at the beginning of an expression. Report that as an error, but also parse and discard a right-hand operand with the appropriate precedence.

The grammar for error productions now looks like this:

```
expression    → comma;          // CHALLENGE!!
comma         → ternary ("," ternary)*;
ternary       → ternary "?" ternary ":" ternary;
               | equality;
equality      → comparison? ( ( "!=" | "==" ) comparison )* ;
comparison    → term? ( ( ">" | ">=" | "<" | "<=" ) term )* ;
term          → factor? ( ( "-" | "+" ) factor )* ;
factor        → unary? ( ( "/" | "*" ) unary )* ;
unary         → ( "!" | "-" ) unary
               | primary ;
primary       → NUMBER | STRING | "true" | "false" | "nil"
               | "(" expression ")" ;
```

However, when implementing, we defer the check all the down to the `primary` method because it allows us to handle it as an error.

```
private Expr primary() {
    if (match(FALSE)) return new Expr.Literal(false);
    if (match(TRUE)) return new Expr.Literal(true);
    if (match(NIL)) return new Expr.Literal(null);

    if (match(NUMBER, STRING)) {
        return new Expr.Literal(previous().literal);
    }
}
```

```

    }

    if (match(LEFT_PAREN)) {
        Expr expr = expression();
        consume(RIGHT_PAREN, "Expect ')' after expression.");
        return new Expr.Grouping(expr);
    }

    if (match(PLUS, MINUS, STAR, SLASH,
              BANG_EQUAL, EQUAL_EQUAL, GREATER,
              GREATER_EQUAL, LESS, LESS_EQUAL)) {
        error(previous(), "Missing left operand");
        term();
        return null;
    }

    throw error(peek(), "Expect expression.");
}

```